Number: P3125R1
Date: 2024-10-16
Audience: SG1 & LEWG
Reply-to: Hana Dusíková

Table of contents

constexpr pointer tagging

This paper proposes a new library non-owning pointer type to semantically represent tagged pointer of various schemes. This standardizes existing practice which currently can't be expressed in C++ without triggering undefined behaviour. This proposal also makes this technique available during constant evaluation which allows number of use-cases to be implementable in `constexpr`.

Revision history

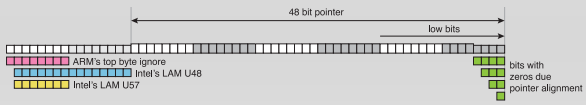
- R0 → R1: proposing and explaining design based on existing implementation

Introduction and motivation

Pointer tagging is widely known and used technique (Glasgow Haskell Compiler, LLVM's `PointerIntPair`, `PointerUnion`, CPython's garbage collector, Objective C / Swift, Chrome's V8 JavaScript engine, GAP, OCaml, PBRT). All major CPU vendors provides mechanism for pointer tagging (Intel's LAM linear address masking, AMD's Upper Address Ignore, ARM's TBI top byte ignore and MTE memory tagging extension). All widely used 64 bit platforms are not using more than 48 or 49 bits of the pointers.

This functionality widely supported can't be expressed in a standard conforming way.

Rust, Dlang, or Zig has an interface for pointer tagging. This is demonstrating demand for the feature and C++ should have it too and it should also work in `constexpr`.



Generally programmer can consider low-bits used for alignment to be safely used for storing an information. Upper bits are available on different platforms under different conditions